GraphQL Practice - NestJS (Schema-first + Code-first)

✅ Checkpoint A: Schema-First Implementation

Current Status

- ✅ GraphQL packages installed
- ✅ Category and Product services/modules created
- ✅ Schema-first GraphQL schema defined in `src/graphql/schema/shop.graphql`
- ✅ Schema-first resolvers implemented
- ✅ GraphQL module configured in AppModule (schema-first)

How to Test Checkpoint A

Server should be running at: http://localhost:3000/graphql

Query Example:

```
query {
  products {
    id
    name
    price
    category { id name }
  }
}
```

Mutation Example:

```
mutation {
  createCategory(name: "Laptop") { id name }
}
```

Query Products by Category:

```
query {
  productsByCategory(categoryId: "1") {
    id
    name
    price
    category { id name }
  }
}
```

🔄 Switching to Code-First (Checkpoint B)

Steps to Switch:

1. Comment out schema-first providers in `src/graphql/graphql.module.ts`
2. Uncomment code-first providers in `src/graphql/graphql.module.ts`
3. In `src/app.module.ts`, comment out `typePaths` and uncomment `autoSchemaFile`
4. Restart the server (npm run start:dev)
5. Schema will be auto-generated at `src/graphql/schema.gql`

What's New in Code-First:

- Types defined with TypeScript classes using `@ObjectType()` and `@Field()` decorators
- Inputs defined with `@InputType()` decorator
- Schema automatically generated from code
- Better type safety in resolvers

Files Structure

Schema-First

- `src/graphql/schema/shop.graphql` - GraphQL schema definition
- `src/graphql/resolvers/category.resolver.ts` - Category resolver
- `src/graphql/resolvers/product.resolver.ts` - Product resolver

Code-First

- `src/graphql/types/category.type.ts` - Category ObjectType
- `src/graphql/types/product.type.ts` - Product ObjectType
- `src/graphql/inputs/create-product.input.ts` - CreateProduct InputType
- `src/graphql/resolvers/category.codefirst.resolver.ts` - Category code-first resolver
- `src/graphql/resolvers/product.codefirst.resolver.ts` - Product code-first resolver

Key Differences

Schema-First vs Code-First

Aspect	Schema-First	Code-First
Schema Definition	`.graphql` files	TypeScript classes
Type Definition	GraphQL Schema Language	Decorators (@ObjectType, @Field)
Generated	Resolvers from schema	Schema from code
IDE Support	GraphQL schema support	Full TypeScript support
Refactoring	Changes in .graphql + code	Single point of change

Aspect	Schema-First	Code-First
Learning Curve	Schema language first	TypeScript-centric

Reflection Questions

1. **In schema-first, what happens if resolver name doesn't match schema?**
    - GraphQL won't be able to resolve that field/query, and the resolver will never be called.
2. **In code-first, where do types and inputs come from?**
    - They're defined using TypeScript classes with decorators (@ObjectType, @InputType, @Field).
3. **Which approach is easier for frontend team collaboration?**
    - Schema-first can be better if you want to design API contract first before implementation.
    - Code-first is better if you have a single team working on both backend and frontend.
4. **Which approach is easier for refactoring in TypeScript?**
    - Code-first is easier because changes are in one place (the TypeScript class).
    - With schema-first, you need to update both the .graphql file and the resolver.